

Position Prediction

Gao

2024-03-05

Introduction

The NHL is one of the most elite sports leagues in the world, and deals often hit 8 digit figures. Thus it is important for each team to put each player in the position they play best, which this paper aims to do. The positions players can play are as follows: center, defense and wing. The goalie position has been removed because it is fundamentally different from the rest of the positions, and the side distinction has been removed for wings since there is no theoretical difference between each wing. The dataset this model is built off of consists of every NHL recruit with adequate data from 2004 to 2021. The data has been trimmed down to only include the player's year recruited, their pick number, the team they were drafted by, their name, nationality, position (which we are trying to predict), age, games played, goals, assists, points, point differential, and total penalty minutes.

Libraries are imported

```
library(tidymodels)
```

```
## -- Attaching packages ----- tidymodels 1.1.1 --
```

```
## v broom          1.0.5      v recipes          1.0.10
## v dials          1.2.1      v rsample          1.2.0
## v dplyr          1.1.4      v tibble          3.2.1
## v ggplot2        3.5.0      v tidyr           1.3.1
## v infer          1.0.6      v tune            1.1.2
## v modeldata      1.3.0      v workflows       1.1.4
## v parsnip        1.2.0      v workflowsets    1.0.1
## v purrr          1.0.2      v yardstick       1.3.0
```

```
## -- Conflicts ----- tidymodels_conflicts() --
```

```
## x purrr::discard() masks scales::discard()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## x recipes::step()  masks stats::step()
## * Search for functions across packages at https://www.tidymodels.org/find/
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v forcats 1.0.0      v readr 2.1.5
## v lubridate 1.9.3    v stringr 1.5.1
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x readr::col_factor() masks scales::col_factor()
## x purrr::discard()   masks scales::discard()
## x dplyr::filter()    masks stats::filter()
## x stringr::fixed()   masks recipes::fixed()
## x dplyr::lag()        masks stats::lag()
## x readr::spec()       masks yardstick::spec()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(rpart.plot)
```

```
## Loading required package: rpart
##
## Attaching package: 'rpart'
##
## The following object is masked from 'package:dials':
##
##   prune
```

```
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following objects are masked from 'package:yardstick':
##
##   precision, recall, sensitivity, specificity
##
## The following object is masked from 'package:purrr':
##
##   lift
```

```
library(dplyr)
```

Datset is imported

```
NHL <- read_csv("nhldraft.csv")
```

```
## Rows: 1319 Columns: 14
## -- Column specification -----
## Delimiter: ","
## chr (4): Team, Player, Nationality, Position
## dbl (10): ID, Year, Pick, Age, GamesPlayed, Goals, Assists, Points, PlusMinu...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

Convert the Position column from the character type to the factor type. This is necessary because the dependent variable in a classification model must be of the factor type.

```
NHL$Position <- as.factor(NHL$Position)
```

Split the data into testing and training sets using a 75-25 split

```
NHL_split <- initial_split(NHL, prop = 0.75)
train <- training(NHL_split)
test <- testing(NHL_split)
```

Create a decision tree model with a depth of 30 and a minimum of 1 data points per node

```
dtree_mod <- decision_tree(mode = "classification", min_n = 1, tree_depth = 30)
```

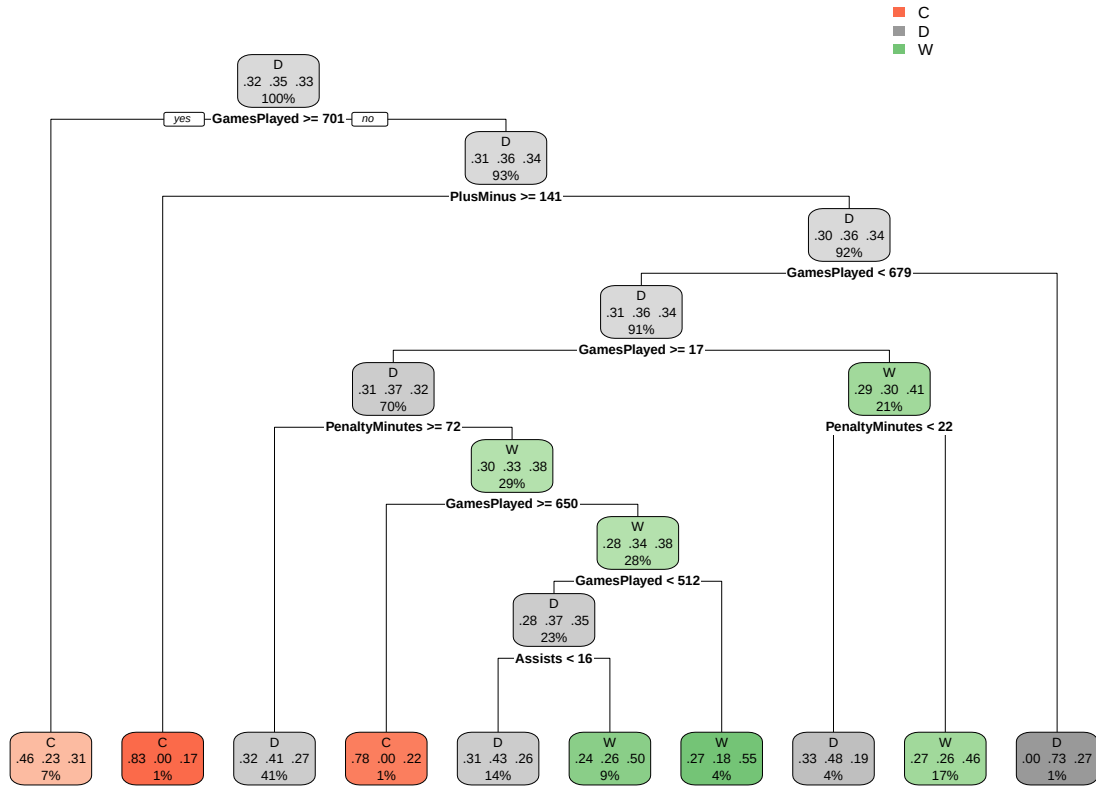
Fit the model to predict the Position variable given all of the relevant remaining variables in the data set.

```
dtree_mod_fit <- fit(object = dtree_mod, formula = Position ~ Age + Goals + Assists + Points + PlusMinus)
```

Plot the model.

```
rpart.plot(extract_fit_engine(dtree_mod_fit))
```

```
## Warning: Cannot retrieve the data used to build the model (so cannot determine roundint and is.binary)
## To silence this warning:
##   Call rpart.plot with roundint=FALSE,
##   or rebuild the rpart model with model=TRUE.
```



Create the predictions for the model using the test data.

```
predictions <- predict(dtree_mod_fit, new_data = test)

predictions
```

```
## # A tibble: 330 x 1
##   .pred_class
##   <fct>
## 1 W
## 2 D
## 3 C
## 4 W
## 5 W
## 6 D
## 7 W
## 8 W
## 9 W
## 10 D
## # i 320 more rows
```

View the confusion matrix for the model as well as the accuracy and kappa metrics. Note that the kappa metric is extremely low, indicating that this model is not very accurate. This makes sense as it will likely be very difficult to predict the position a player plays from their statistics alone. This is especially true with the offensive positions, as at the level of the NHL they will all have similar opportunities to score. For example, Howe and Jagr played/play wing, Gretzky and Messier played center. These 4 players are the top 4 point leaders in the NHL. Defensemen should have lower goals/assists than the offensive positions, which is reflected by the decision tree's choice of assigning players with lower goal counts to defense.

```
confusionMatrix(data = predictions$.pred_class, reference = test$Position)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  C  D  W
##           C  9 17 11
##           D 76 58 66
##           W 27 40 26
##
## Overall Statistics
##
##           Accuracy : 0.2818
##           95% CI : (0.2339, 0.3337)
##           No Information Rate : 0.3485
##           P-Value [Acc > NIR] : 0.9958
##
##           Kappa : -0.0836
##
## Mcnemar's Test P-Value : 6.117e-11
##
## Statistics by Class:
##
##           Class: C Class: D Class: W
## Sensitivity      0.08036  0.5043  0.25243
## Specificity      0.87156  0.3395  0.70485
## Pos Pred Value   0.24324  0.2900  0.27957
## Neg Pred Value   0.64846  0.5615  0.67511
## Prevalence       0.33939  0.3485  0.31212
## Detection Rate   0.02727  0.1758  0.07879
## Detection Prevalence 0.11212  0.6061  0.28182
## Balanced Accuracy 0.47596  0.4219  0.47864
```

Check the variable importance of the model. Make a note of any variables that have an importance of 0.

As we can see Age is the only variable that has an importance of 0

```
dtree_mod_fit %>% extract_fit_engine() %>% varImp()
```

```
##           Overall
## Age          15.27125
## Assists      18.44329
## GamesPlayed  20.67858
## Goals        11.51677
## PenaltyMinutes 14.94981
## PlusMinus     8.08850
## Points       13.17257
```

Create six cross validation folds for the training data

```
folds <- vfold_cv(train, v = 6)
```

Create an automatic grid of hyperparameters for the model

```
tree_grid_auto <- grid_regular(tree_depth(), min_n(), cost_complexity())
```

```
tree_grid_auto
```

```
## # A tibble: 27 x 3
##   tree_depth min_n cost_complexity
##       <int> <int>         <dbl>
## 1         1     2  0.0000000001
## 2         8     2  0.0000000001
## 3        15     2  0.0000000001
## 4         1    21  0.0000000001
## 5         8    21  0.0000000001
## 6        15    21  0.0000000001
## 7         1    40  0.0000000001
## 8         8    40  0.0000000001
## 9        15    40  0.0000000001
## 10        1     2  0.00000316
## # i 17 more rows
```

```
print(tree_grid_auto, n = 27)
```

```
## # A tibble: 27 x 3
##   tree_depth min_n cost_complexity
##       <int> <int>         <dbl>
## 1         1     2  0.0000000001
## 2         8     2  0.0000000001
## 3        15     2  0.0000000001
## 4         1    21  0.0000000001
## 5         8    21  0.0000000001
## 6        15    21  0.0000000001
## 7         1    40  0.0000000001
## 8         8    40  0.0000000001
## 9        15    40  0.0000000001
## 10        1     2  0.00000316
## 11        8     2  0.00000316
## 12       15     2  0.00000316
## 13        1    21  0.00000316
## 14        8    21  0.00000316
## 15       15    21  0.00000316
## 16        1    40  0.00000316
## 17        8    40  0.00000316
## 18       15    40  0.00000316
## 19        1     2  0.1
## 20        8     2  0.1
## 21       15     2  0.1
## 22        1    21  0.1
## 23        8    21  0.1
## 24       15    21  0.1
## 25        1    40  0.1
## 26        8    40  0.1
## 27       15    40  0.1
```

Use the grid and the folds to tune the model

```
dtree_mod_tune <- decision_tree(mode = "classification", tree_depth = tune(), min_n = tune(), cost_complexity = tune())
tuning_data <- tune_grid(dtree_mod_tune, preprocessor = Position ~ Goals + Assists + Points + PlusMinus)
print(tuning_data %>% collect_metrics, n = 54)
```

```
## # A tibble: 54 x 9
##   cost_complexity tree_depth min_n .metric .estimator mean n std_err
##   <dbl> <int> <int> <chr> <chr> <dbl> <int> <dbl>
## 1 0.0000000001 1 2 accuracy multiclass 0.348 6 0.0105
## 2 0.0000000001 1 2 roc_auc hand_till 0.498 6 0.00451
## 3 0.0000000001 8 2 accuracy multiclass 0.315 6 0.0119
## 4 0.0000000001 8 2 roc_auc hand_till 0.471 6 0.0114
## 5 0.0000000001 15 2 accuracy multiclass 0.322 6 0.0114
## 6 0.0000000001 15 2 roc_auc hand_till 0.490 6 0.0101
## 7 0.0000000001 1 21 accuracy multiclass 0.346 6 0.00909
## 8 0.0000000001 1 21 roc_auc hand_till 0.496 6 0.00493
## 9 0.0000000001 8 21 accuracy multiclass 0.328 6 0.0171
## 10 0.0000000001 8 21 roc_auc hand_till 0.477 6 0.0139
## 11 0.0000000001 15 21 accuracy multiclass 0.328 6 0.0125
## 12 0.0000000001 15 21 roc_auc hand_till 0.486 6 0.0119
## 13 0.0000000001 1 40 accuracy multiclass 0.346 6 0.00909
## 14 0.0000000001 1 40 roc_auc hand_till 0.496 6 0.00493
## 15 0.0000000001 8 40 accuracy multiclass 0.320 6 0.0160
## 16 0.0000000001 8 40 roc_auc hand_till 0.476 6 0.0126
## 17 0.0000000001 15 40 accuracy multiclass 0.336 6 0.0173
## 18 0.0000000001 15 40 roc_auc hand_till 0.487 6 0.0149
## 19 0.00000316 1 2 accuracy multiclass 0.348 6 0.0105
## 20 0.00000316 1 2 roc_auc hand_till 0.498 6 0.00451
## 21 0.00000316 8 2 accuracy multiclass 0.315 6 0.0119
## 22 0.00000316 8 2 roc_auc hand_till 0.471 6 0.0114
## 23 0.00000316 15 2 accuracy multiclass 0.322 6 0.0114
## 24 0.00000316 15 2 roc_auc hand_till 0.490 6 0.0101
## 25 0.00000316 1 21 accuracy multiclass 0.346 6 0.00909
## 26 0.00000316 1 21 roc_auc hand_till 0.496 6 0.00493
## 27 0.00000316 8 21 accuracy multiclass 0.328 6 0.0171
## 28 0.00000316 8 21 roc_auc hand_till 0.477 6 0.0139
## 29 0.00000316 15 21 accuracy multiclass 0.328 6 0.0125
## 30 0.00000316 15 21 roc_auc hand_till 0.486 6 0.0119
## 31 0.00000316 1 40 accuracy multiclass 0.346 6 0.00909
## 32 0.00000316 1 40 roc_auc hand_till 0.496 6 0.00493
## 33 0.00000316 8 40 accuracy multiclass 0.320 6 0.0160
## 34 0.00000316 8 40 roc_auc hand_till 0.476 6 0.0126
## 35 0.00000316 15 40 accuracy multiclass 0.336 6 0.0173
## 36 0.00000316 15 40 roc_auc hand_till 0.487 6 0.0149
## 37 0.1 1 2 accuracy multiclass 0.350 6 0.0104
## 38 0.1 1 2 roc_auc hand_till 0.5 6 0
## 39 0.1 8 2 accuracy multiclass 0.350 6 0.0104
## 40 0.1 8 2 roc_auc hand_till 0.5 6 0
## 41 0.1 15 2 accuracy multiclass 0.350 6 0.0104
## 42 0.1 15 2 roc_auc hand_till 0.5 6 0
## 43 0.1 1 21 accuracy multiclass 0.350 6 0.0104
```

```

## 44    0.1          1    21 roc_auc hand_till 0.5      6 0
## 45    0.1          8    21 accuracy multiclass 0.350  6 0.0104
## 46    0.1          8    21 roc_auc hand_till 0.5      6 0
## 47    0.1         15    21 accuracy multiclass 0.350  6 0.0104
## 48    0.1         15    21 roc_auc hand_till 0.5      6 0
## 49    0.1          1    40 accuracy multiclass 0.350  6 0.0104
## 50    0.1          1    40 roc_auc hand_till 0.5      6 0
## 51    0.1          8    40 accuracy multiclass 0.350  6 0.0104
## 52    0.1          8    40 roc_auc hand_till 0.5      6 0
## 53    0.1         15    40 accuracy multiclass 0.350  6 0.0104
## 54    0.1         15    40 roc_auc hand_till 0.5      6 0
## # i 1 more variable: .config <chr>

```

Display the best hyperparameter values for the model

The best hyperparameter values are `min_n = 40`, and `tree_depth = 8`

```
tuning_data %>% select_best(metric = "accuracy")
```

```
## # A tibble: 1 x 4
##   cost_complexity tree_depth min_n .config
##         <dbl>         <int> <int> <chr>
## 1           0.1             1     2 Preprocessor1_Model19
```

```
tuning_data %>% show_best(metric = "accuracy")
```

```
## # A tibble: 5 x 9
##   cost_complexity tree_depth min_n .metric .estimator mean      n std_err
##         <dbl>         <int> <int> <chr>    <chr>    <dbl> <int>  <dbl>
## 1           0.1             1     2 accuracy multiclass 0.350     6 0.0104
## 2           0.1             8     2 accuracy multiclass 0.350     6 0.0104
## 3           0.1            15     2 accuracy multiclass 0.350     6 0.0104
## 4           0.1             1    21 accuracy multiclass 0.350     6 0.0104
## 5           0.1             8    21 accuracy multiclass 0.350     6 0.0104
## # i 1 more variable: .config <chr>
```

Create the final model that uses the best hyperparameter values.

```
dtree_mod_final <- decision_tree(mode = "classification", min_n = 21, tree_depth = 8)
```

Fit the final model without using any variables that have an importance of 0.

```
dtree_mod_fit_final <- fit(object = dtree_mod, formula = Position ~ Goals + Assists + Points + PlusMinus)
```

Use the final model to create predictions.

```
predictions_final <- predict(dtree_mod_fit_final, new_data = test)
```

```
predictions_final
```

```
## # A tibble: 330 x 1
##   .pred_class
##   <fct>
## 1 W
## 2 D
## 3 C
## 4 W
## 5 W
## 6 D
## 7 W
## 8 W
## 9 W
## 10 D
## # i 320 more rows
```

View the confusion matrix for the final model. Note that the kappa value has not changed, and that the predictions are the same as before. This is likely because there are so many different players with statistics ranging across so many games that it becomes difficult to pinpoint a player's position. A better way to approach this problem would be to normalize all values, points, plusminus, penalty minutes, etc. across all games by dividing each value by the number of games played. This will allow for offense and defense to better distinguish from each other, as well as preventing players that played large numbers of games from skewing the data.

```
confusionMatrix(data = predictions_final$.pred_class, reference = test$Position)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  C  D  W
##           C  9 17 11
##           D 76 58 66
##           W 27 40 26
##
## Overall Statistics
##
##           Accuracy : 0.2818
##           95% CI : (0.2339, 0.3337)
##           No Information Rate : 0.3485
##           P-Value [Acc > NIR] : 0.9958
##
##           Kappa : -0.0836
##
## Mcnemar's Test P-Value : 6.117e-11
##
## Statistics by Class:
##
##           Class: C Class: D Class: W
## Sensitivity      0.08036  0.5043  0.25243
## Specificity      0.87156  0.3395  0.70485
## Pos Pred Value   0.24324  0.2900  0.27957
## Neg Pred Value   0.64846  0.5615  0.67511
## Prevalence       0.33939  0.3485  0.31212
## Detection Rate   0.02727  0.1758  0.07879
## Detection Prevalence 0.11212  0.6061  0.28182
## Balanced Accuracy 0.47596  0.4219  0.47864
```